Itinerario: Lenguajes de Programación Web FUNDAE

https://www.efundae.es/course/view.php?id=906

Este recorrido te llevará a dominar las tecnologías clave del desarrollo web tales como HTML, CSS, JavaScript (Frontend), PHP, Java, Python, Node, C# y .NET (Backend).

Guía de aprendizaje

Itinerarios

Este curso forma parte de un itinerario con niveles del 3 al 6. Están enumerados de esta manera, porque son una referencia específica a niveles intermedios y avanzados de competencias digitales del marco de la Comisión Europea.

- Nivel 3. Lenguajes de programación web: HTML, CSS, JavaScript (frontend)
- Nivel 4. Lenguajes de programación web: PHP, Java, Python, Node, C#, .NET (backend)
- Nivel 5. Lenguajes de programación web: HTML, CSS, JavaScript (frontend)
- Nivel 6. Lenguajes de programación web: PHP, Java, Python, Node, C#, .NET (backend)

Código Samurái

Código Samurái es un programa de formación en competencias digitales de programación que consta de seis itinerarios:

1. Lenguajes de programación web: HTML, CSS, JavaScript (frontend), PHP, Java, Python, Node, C#, .NET (backend)
2. Sistemas Cloud (Amazon, Google y Microsoft)
3. Sistemas Gestores de Bases de Datos (SGBD): SQL Server, MySQL, Oracle, MaríaDB, mongoDB, otros)
4. Desarrollo web/frameworks: Angular JS, React, Spring, Hibernate, Struts, .NET, Synphony, Zend, Yii, Node.js, JQuery, Ajax, otros.
5. Desarrollo de Aplicaciones web, multiplataforma (Flutter, Dart).
6. Desarrollo de APP (en Android, IOS)

NIVEL 3. FRONTEND

HTML, CSS y Javascript

¿Qué es un portfolio digital?

Un portafolio web es una **colección digital de información** sobre tu trabajo, experiencias, calificaciones, habilidades o elogios que has obtenido y deseas compartir.

Da legitimidad a tu trabajo y brinda a los profesionales de la industria una mejor comprensión de lo que puedes hacer.

Ejemplos de portfolios exitosos: https://p5aholic.me/; https://bonhomme.lol/; https://www.edwardh.io/

¿Cómo hacerlo?

1. **Planificación**. Antes de empezar la construcción de tu portfolio, es crucial que decidas qué proyectos y habilidades quieres destacar. Puedes incluir desde proyectos personales hasta ejercicios de formación, como los retos que harás en este curso. También puedes añadir certificaciones. Incluso, puedes darle un toque muy humano si aportas reflexiones personales.
2. **Selección de la plataforma**: elegir dónde construirás tu portfolio. Plataformas como GitHub Pages, WordPress, y Wix ofrecen excelentes puntos de partida con opciones gratuitas y, en alguna, de pago. Tienes los enlaces en esta misma página. Cada plataforma tiene sus fortalezas, dependiendo de lo que necesites. Por ejemplo, GitHub Pages es ideal para mostrar proyectos de código.
3. **Diseño**: El diseño de tu portfolio debe ser limpio y profesional. Todas las plataformas tienen plantillas gratuitas. Elige una plantilla que hable de quién eres y asegúrate de que tu sitio sea fácil de navegar. También puedes crear tú la plantilla. O incluso puedes tener tu propia web que te sirva como muestra. Pero si quieres ir a lo seguro, te aconsejamos que utilices, al menos de inicio, una plantilla.

   https://github.com/umutxyp/Personal-Website

   El diseño y las plantillas de Github son muy funcionales, todas las páginas son muy similares. Si eres programador, o quieres desarrollar competencias digitales de ese área, esta es la página de referencia en todo el mundo. En ella compartirás el código que generes. Incluso puedes tener un portfolio en otra plataforma y un enlace a tu Github.

   Por otra parte, para incluir más diseño, más experiencia de usuario, más personalidad, te aconsejamos WordPress o Wix.
4. **Creación del contenido**: Para cada proyecto, incluye una descripción detallada, lo que aprendiste y, por supuesto, imágenes o enlaces que enriquezcan tu presentación. O el código que has utilizado, para que cualquiera pueda comprobar lo útil y eficiente que es. Promueve tu portfolio, que otros compañeros y profesionales lo vean y te den feedback. Sus opiniones son oro puro para pulir tu trabajo.
5. **Publica y actualiza** tu portfolio regularmente. Un buen portfolio no solo está bien hecho, sino que también está bien mantenido.

¿Por qué tener un portfolio para esta formación? Porque cada reto y proyecto que realices es una oportunidad de demostrar tus habilidades. Sube esos proyectos y muestra al mundo lo capaz que eres.

Para crear un portfolio digital que albergue evidencias de aprendizaje de un programa formativo de análisis y gestión de datos, existen varias herramientas gratuitas interesantes:

- GitHub Pages (https://pages.github.com/) GitHub Pages es ideal para portfolios para programadores: - Permite alojar sitios web estáticos directamente desde un repositorio de GitHub. - Es gratuito y ofrece hosting ilimitado para proyectos públicos. - Facilita mostrar proyectos de código, análisis de datos y visualizaciones. Permite versionar el contenido del portfolio. - Es muy valorado en el ámbito de la ciencia de datos y programación.

Usuario: estdiesai83@gmail.com

Password: 172223Es@

- Google Sites (https://sites.google.com/) - Permite crear fácilmente páginas web sin necesidad de conocimientos técnicos. - Se integra con otras herramientas de Google como Docs, Sheets y Drive para incluir documentos y archivos. - Ofrece plantillas prediseñadas que facilitan la creación del portfolio. - Permite colaboración en tiempo real si se trabaja en equipo. - Es accesible desde cualquier dispositivo con conexión a internet. 11
- WordPress (https://wordpress.com/es/) - Ofrece una versión gratuita con hosting incluido en wordpress.com. - Tiene numerosas plantillas y temas optimizados para portfolios. - Permite personalizar el diseño y funcionalidades. - Cuenta con una gran comunidad de usuarios y recursos de ayuda. - Es escalable si en el futuro se quiere migrar a un hosting propio.
- Behance (https://www.behance.net/) Behance es una plataforma especializada en portfolios creativos: - Está enfocada en mostrar proyectos visuales y creativos. - Permite organizar los trabajos en proyectos y colecciones. - Ofrece estadísticas sobre las visitas al portfolio. - Facilita la conexión con otros profesionales del sector. - Es especialmente útil si el programa incluye visualización de datos

1. Introducción a la programación web

1.1 Conceptos básicos de la web y arquitectura cliente-servidor

Frontend es la parte del desarrollo que ven los usuarios, la cara que muestra la aplicación.

El **backend** es el “alma”: la lógica, el servidor, la aplicación y la base de datos de la página.

Y ambas se relacionan a través de las APIs, intercambiando solicitudes del frontend y datos del backend.

Las redes permiten que las computadoras se comuniquen entre sí, enviando y recibiendo datos entre ellas. Esto hace posible trabajar de forma remota, utilizar las redes sociales, hacer videoconferencias o que participes en este curso.

- **¿Qué es la World Wide Web?**

  La World Wide Web (WWW) es un sistema de **documentos y recursos** web interconectados mediante hipervínculos y accesibles a través de Internet.

  Tim Berners-Lee, un científico de la computación que trabajaba en el CERN, creó la web en 1989. Su objetivo era crear un **espacio de información global** que permitiera a las personas compartir conocimientos y comunicarse independientemente de su ubicación geográfica.
- **Sistema de hipertexto**

  La propuesta inicial de Berners-Lee incluía la creación de un **sistema de hipertexto** que utilizara Internet como medio de transmisión.
- **Evolución de la web**

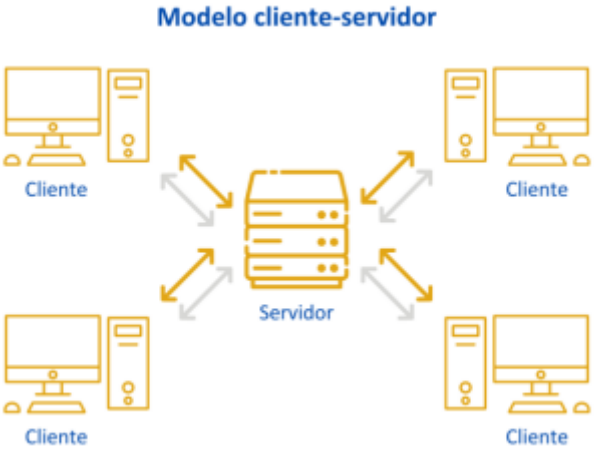
La web ha evolucionado desde simples páginas de texto en los primeros días del navegador Mosaic, a las complejas aplicaciones web de hoy en día que permiten interacciones en tiempo real, transacciones seguras, y experiencias multimedia ricas.

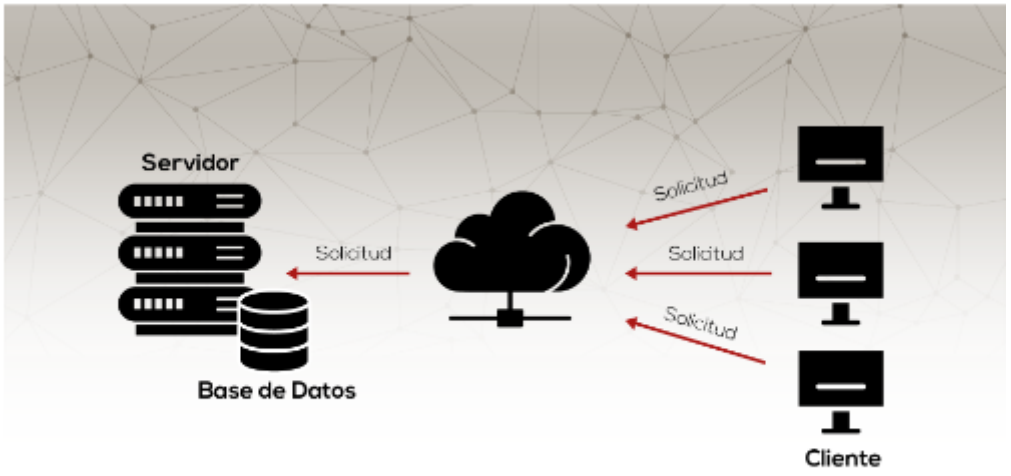
Esta evolución ha sido posible gracias al desarrollo continuo de tecnologías web, estándares y protocolos.

Arquitectura cliente-servidor

La arquitectura cliente-servidor es un **marco de diseño de software** en el que las tareas se reparten entre los proveedores de recursos o servicios, conocidos como servidores, y los solicitantes de servicios, conocidos como clientes.

Los clientes realizan solicitudes a los servidores, que luego procesan la solicitud y devuelven la respuesta adecuada. Esta arquitectura es fundamental para el funcionamiento de la web, permitiendo la **comunicación y transferencia de datos** entre sistemas dispersos geográficamente.





La implementación de esta idea dio origen a tres tecnologías fundamentales que forman la base de la web actual:

- **HTML** (Lenguaje de Marcado de Hipertexto): Diseñado para estructurar y presentar contenido en la web.
- **URI** (Identificador de Recursos Uniforme): Permite identificar de manera única recursos en la web.
- **HTTP** (Protocolo de Transferencia de Hipertexto): Define cómo se solicitan y transmiten los recursos entre clientes y servidores.

Funcionamiento y componentes

Cliente

En el contexto de la web, el cliente suele ser un **navegador web** que envía solicitudes al servidor para acceder a páginas de información. Las páginas web que se reciben se transfieren con el protocolo HTTP. El cliente interpreta el contenido recibido del servidor y lo muestra al usuario. Si lo que se recibe son objetos como vídeo, imágenes, o cualquier otro recurso, el cliente debe activar aplicaciones externas que puedan gestionarlo, como los navegadores.

Servidor

El servidor es un **sistema de software o hardware** que responde a las solicitudes del cliente a través de la red. Los servidores web almacenan los recursos web, como archivos HTML, CSS, y JavaScript, y los envían a los clientes según sea necesario.

Funciona de la siguiente manera: si encuentra en su sistema de ficheros el documento HTML solicitado por el cliente, lo envía y cierra la conexión; en caso contrario, envía un **código de error** que cierra la conexión.

La comunicación entre clientes y servidores se realiza utilizando el protocolo **HTTP**. Cuando un usuario escribe una URL en su navegador, este envía una solicitud HTTP al servidor correspondiente. El servidor procesa la solicitud y envía una respuesta HTTP al cliente, que puede ser una página web, un mensaje de error, o cualquier otro recurso web.

Importancia para profesionales de la programación
Entender la arquitectura cliente-servidor es crucial para desarrolladores web, ya que proporciona la base para diseñar y construir aplicaciones web eficientes y seguras. Los profesionales deben comprender cómo optimizar las solicitudes y respuestas entre el cliente y el servidor, cómo gestionar las sesiones de usuario, y cómo asegurar la comunicación contra amenazas cibernéticas.

1.2 Navegadores web y herramientas de desarrollo

Los navegadores web son **herramientas muy utilizadas** en nuestra vida cotidiana, pues nos permiten el acceso y la navegación por la World Wide Web.

Estos programas interpretan el código de las páginas web (principalmente HTML, CSS, y JavaScript) y lo presentan visualmente al usuario.

Origen:

Tim Berners-Lee creador de la World Wide Web (www) creó el primer navegador con el mismo nombre, más tarde lo llamaron Nexus. En 1993, Marc Andreessen lideró el desarrollo de Mosaic, el primer navegador gráfico, en el NCSA.

Mosaic hizo que la web fuera más accesible para el público general al soportar imágenes inline con el texto, lo que era una novedad en ese momento. La popularidad de Mosaic llevó al desarrollo de Netscape Navigator en 1994, que dominó el mercado hasta la llegada de Internet Explorer de Microsoft.

El lanzamiento de Mozilla Firefox en 2004 y de Google Chrome en 2008 marcó el inicio de una nueva era para los navegadores web, que se centraron en la velocidad, la seguridad y el soporte de estándares web.

Chrome, en particular, jugó un papel crucial en la promoción de innovaciones como las aplicaciones web progresivas (PWA) y la mejora de la experiencia del usuario.

Actualmente:

Los tres principales navegadores web del mercado, **Chrome, Firefox y Edge,** ofrecen características distintivas que los hacen únicos y los posicionan como líderes en el ámbito de la navegación web.

1. **Google Chrome**

   Es el **navegador más popular** a nivel mundial debido a su velocidad, eficiencia y el amplio soporte para las últimas tecnologías web. La integración con el ecosistema de Google proporciona una experiencia de usuario fluida y personalizada, permitiendo la sincronización de marcadores, contraseñas y extensiones a través de dispositivos.

   La integración con el ecosistema de Google proporciona una experiencia de usuario fluida y personalizada, permitiendo la sincronización de marcadores, contraseñas y extensiones a través de dispositivos. Chrome es conocido por su vasta biblioteca de extensiones disponibles en **Chrome Web Store**, lo que permite a los usuarios personalizar su experiencia de navegación según sus necesidades. Además, Chrome incluye características de seguridad avanzadas como la generación automática de contraseñas fuertes y alertas de violación de datos. Sin embargo, es conocido por su consumo de recursos, especialmente la memoria RAM.

2. **Mozilla Firefox**

   Se distingue por su **enfoque en la privacidad y seguridad** del usuario, ofreciendo protecciones avanzadas contra el rastreo y una gestión de contraseñas eficiente.

   Firefox Quantum, su actualización significativa, **mejora el rendimiento** y la velocidad del navegador aprovechando mejor los procesadores multinúcleo disponibles en los dispositivos modernos.

   Permite a los usuarios ajustar la interfaz y funcionalidad a través de una amplia gama de extensiones y tema.

   Es el único de los tres principales navegadores que es de **código abierto**, lo que refuerza su compromiso con la transparencia y la colaboración comunitaria en el desarrollo.

3. **Microsoft Edge**

   Microsoft Edge, basado en Chromium desde su relanzamiento en 2020, ofrece una experiencia de **navegación rápida y segura**, con características únicas como el **modo lector** y una mejor visualización de PDF.

Al igual que Chrome, Edge soporta **extensiones** de Chrome Web Store, además de las disponibles en su propia tienda de extensiones, ampliando las funcionalidades para los usuarios.

También ha integrado innovaciones como Bing Chat (anteriormente conocido como Microsoft Copilot), basado en GPT-4, proporcionando una experiencia interactiva avanzada directamente en el navegador. Edge es especialmente atractivo para los usuarios de Windows por su integración con el sistema operativo, ofreciendo una experiencia más cohesiva y optimizada.

Herramientas de desarrollo integradas

Estas herramientas ofrecen una amplia gama de funcionalidades, desde la inspección de elementos HTML y la edición de CSS en tiempo real hasta la depuración de JavaScript y el análisis del rendimiento de la página directamente en el entorno en el que se ejecutan.

Las herramientas de desarrollo son fundamentales para el diseño, la implementación y el mantenimiento eficiente de sitios web y aplicaciones web.

Las tres herramientas de desarrollo principales:

1. **Google Chrome: DevTools**

   Son un conjunto integral de herramientas para desarrolladores que se integran directamente en el navegador.

   Proporcionan funcionalidades clave como:

   - **Inspección y edición de HTML/CSS:** Permite a los desarrolladores ver y modificar el HTML y el CSS de una página para experimentar con cambios de diseño en tiempo real.
   - **Consola JavaScript**: Ofrece una terminal interactiva para ejecutar scripts, mostrar mensajes de error y depurar código JavaScript.
   - **Rendimiento y perfilado**: Ayuda a identificar cuellos de botella en el rendimiento, analizar el uso de la memoria y optimizar la velocidad de carga de la página.
   - **Red y recursos**: Permite monitorear las solicitudes de red, inspeccionar las cargas de archivos y optimizar la carga de recursos.

2. **Mozilla Firefox: Firefox Developer Tools**

   Ofrece un conjunto similar de herramientas con algunas características únicas orientadas a la privacidad y el rendimiento web:

   - **Grid y Flexbox Inspectors**: Herramientas especializadas para diseñar y depurar sistemas de diseño CSS Grid y Flexbox.
   - **Debugger avanzado**: Un entorno de depuración potente con soporte para puntos de interrupción, inspección de variables y simulación de eventos.
   - **Herramientas de rendimiento**: Incluyen un analizador de rendimiento y un monitor de memoria para optimizar la eficiencia de las aplicaciones web.
   - **Network Monitor**: Visualiza las solicitudes de red con detalles sobre tiempos de carga, tamaños de respuesta y headers HTTP.

3. **Microsoft Edge: Edge Developer Tools**

   Basadas en las mismas tecnologías de Chromium DevTools, ofrecen características avanzadas con integraciones únicas en el ecosistema de Microsoft:

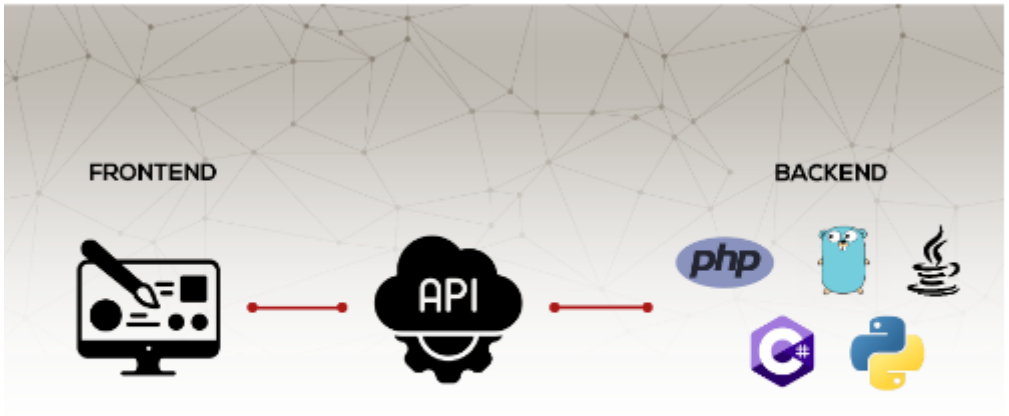
- **Herramientas de compatibilidad**: Facilitan la prueba de compatibilidad de sitios web a través de diferentes versiones de Edge y otros navegadores.
- **Modo de visualización 3D**: Una herramienta única para visualizar y depurar la estructura de elementos de una página en 3D.
- **Herramientas de accesibilidad**: Ayudan a evaluar y mejorar la accesibilidad del sitio web, asegurando que cumpla con los estándares WCAG.
- **Microsoft Clarity Integration**: Proporciona análisis de comportamiento del usuario y mapas de calor dentro del mismo conjunto de herramientas.

1.3 Frontend frente a backend

Frontend y backend son dos componentes críticos que **trabajan en conjunto** para crear aplicaciones web ricas e interactivas.

La distinción entre ambos tipos de desarrollo surgió con la tecnología HTML5, y permite incorporar todo tipo de recursos a una página. Estas dos partes de una aplicación **intercambian información a través de las API**.

Una **API** (interfaz de programación de aplicaciones) permite que diferentes aplicaciones se comuniquen entre sí.



Frontend

El desarrollo frontend se refiere a la parte de la aplicación web que **los usuarios ven y con la que interactúan** directamente. Es el rostro visual de la aplicación, donde la usabilidad, el diseño y la experiencia del usuario son primordiales.

Tecnologías del Frontend

- **HTML** (Lenguaje de Marcado de Hipertexto): La columna vertebral de cualquier página web, estructura el contenido y los elementos visuales en la web.
- **CSS** (Hojas de Estilo en Cascada): Define el estilo y la presentación de los documentos HTML, incluyendo colores, diseños y fuentes.
- **JavaScript** (JS): Añade interactividad a las páginas web, permitiendo la creación de contenido dinámico como formularios en vivo, animaciones y la manipulación de elementos HTML y CSS.

Frameworks y bibliotecas

- **React**: Una biblioteca de JavaScript para construir interfaces de usuario interactivas de manera eficiente.
- **Angular**: Un framework de desarrollo para construir aplicaciones web dinámicas y de una sola página (SPA).
- **Vue.js**: Un framework progresivo para construir interfaces de usuario y aplicaciones de una sola página.

Backend

El desarrollo backend se centra en la "parte trasera" de la aplicación web, el servidor, la aplicación y la base de datos. Es donde **se procesa la lógica de negocio**, se gestionan las solicitudes de los usuarios, y se almacena y recupera la información.

Tecnologías del Backend

- **Lenguajes de Programación**: Python, Ruby, PHP, Java, y Node.js son comunes en el desarrollo backend, cada uno con sus propias bibliotecas y frameworks.
- **Bases de Datos**: Sistemas como MySQL, PostgreSQL, MongoDB, y Firebase para almacenar y gestionar datos.
- **Servidores Web**: Apache y Nginx son ejemplos de software de servidor web que gestionan las solicitudes entrantes de Internet y sirven respuestas a los usuarios.

2.2 Arquitectura y diseño del Backend

- **APIs** (Interfaces de Programación de Aplicaciones): Permiten la comunicación entre diferentes partes de una aplicación o entre distintas aplicaciones.
- **Microservicios**: Una arquitectura que divide una aplicación en una colección de servicios más pequeños, lo que mejora la modularidad y facilita la escalabilidad y el mantenimiento.

Interacción entre frontend y backend

La interacción entre el frontend y el backend es esencial para el funcionamiento de las aplicaciones web. Se realiza principalmente **a través de API**, donde el frontend hace solicitudes al backend, que procesa estas solicitudes y devuelve los datos necesarios para que el frontend los presente al usuario.

Ya hemos visto que el protocolo HTTP es la base de todos los intercambios de información en la web.

Protocolo HTTP El protocolo HTTP (Hypertext Transfer Protocol) es el fundamento de cualquier transferencia de datos en la web, facilitando la comunicación entre cliente y servidor.

Además, tenemos JSON y AJAX:

- **JSON (JavaScript Object Notation)**

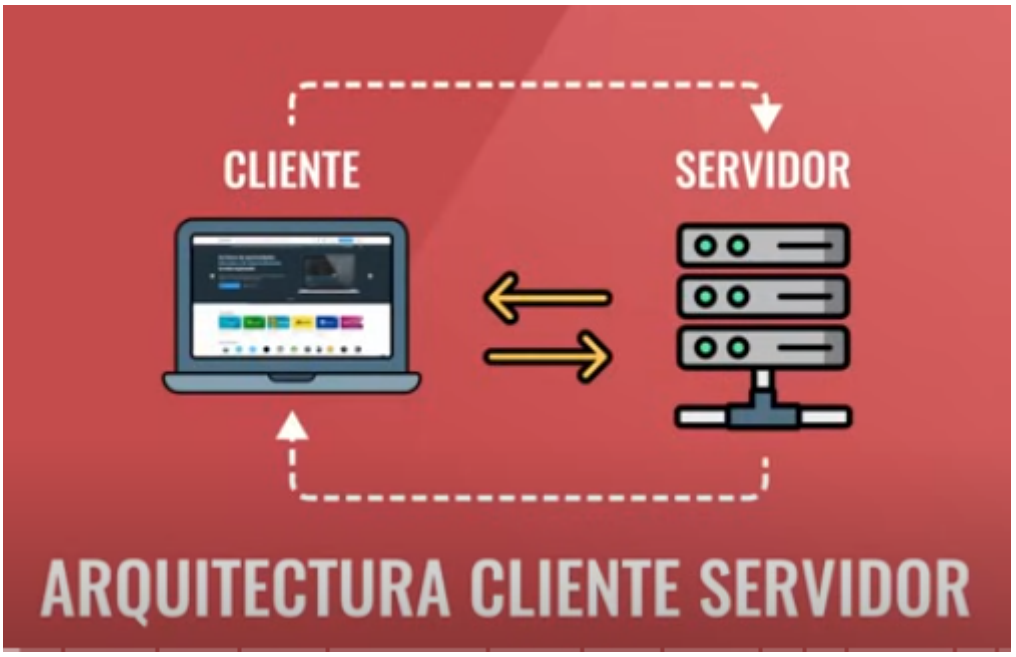
  JSON es un formato ligero de intercambio de datos, fácilmente legible tanto para humanos como para máquinas.

- **AJAX (Asynchronous JavaScript And XML)**

  AJAX permite al frontend realizar solicitudes al backend de manera **asíncrona**, sin necesidad de recargar la página completa.

Ejemplo vídeo Frontend vs Backend:

https://www.youtube.com/watch?v=50RbVujPPGs



El cliente y el servidor son dos máquinas, el cliente es quien pide información y el servidor es quien entrega la información que pide el cliente, siempre y cuando el cliente tenga los permisos, es decir el servidor hace la validación de que el cliente tenga los permisos.

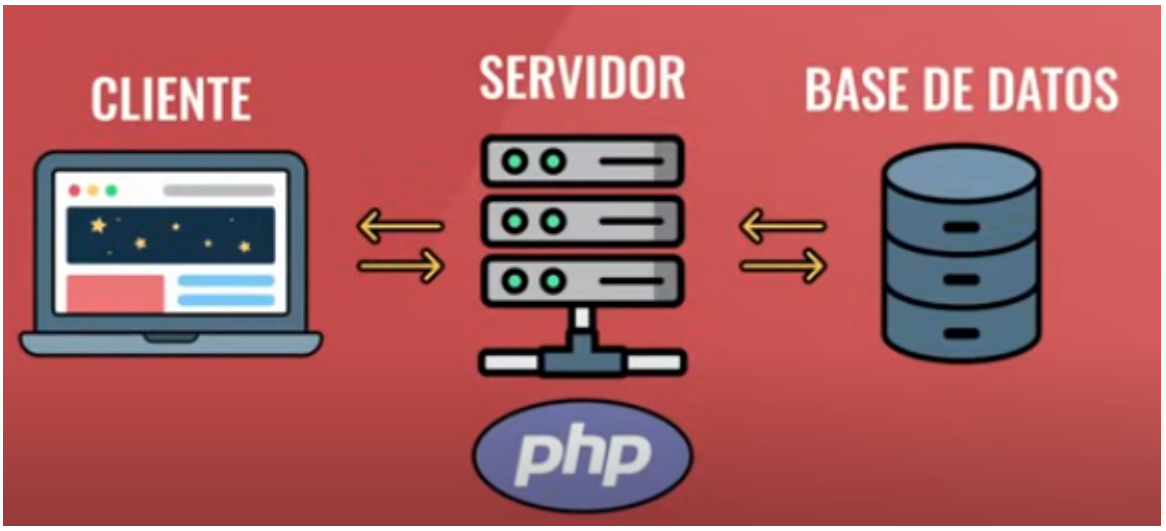
Ej: el cliente entra en una web y a trvés de la conexión a internet esta máquina cliente se conecta con el servidor donde este revisa si tienes los permisos para ver lo que estas solicitando por, ejemplo estas solicitando un video de una clase premium, entonces el servidor verá este usuario tiene acceso o no tiene acceso y según eso, el servidor entregará o no entegrará el video.

Cada solicitud que hace el cliente al servidor se llama **petición**, en el desarrollo web tradicional cada petición al servidor recargaba completamente la página, por ejemplo si tu querias cambiar solamente una foto tenías que traerte toda la página completa de nuevo, no solamente esta foto, pero con tecnologías como **AJAX** o **websockets** podemos hacer peticiones de cosas muy específicas y sólamente cambiar esas cosas en la pantalla haciendo que la web sea mucho mas fluida, más dinámica.

El desarrollo web de forma tradicional:

La web comienza en el año 1992 con html y en esa década aparecen varios lenguajes como Java, Javascript, Python, Php, css que nos ayudan a hacer la web.

Php ha sido el lenguaje más importante en el mundo de la web, al menos en la primera etapa del desarrollo web, aunque Java también fue muy importante en el mundo corporativo, entonces hacíamos desarrollo web principalmente con php, ni siquiera Javascript (hoy en dia es raro), se usaba algunas funciones de jquery, conphp te conectabas a la base de datos, con php creabas las plantillas de lo que se iba a ver en la pantalla, no había una división clara entre programador backend (programador que hace las cosas en el servidor) y programador frontend (el que hace las cosas en el cliente).



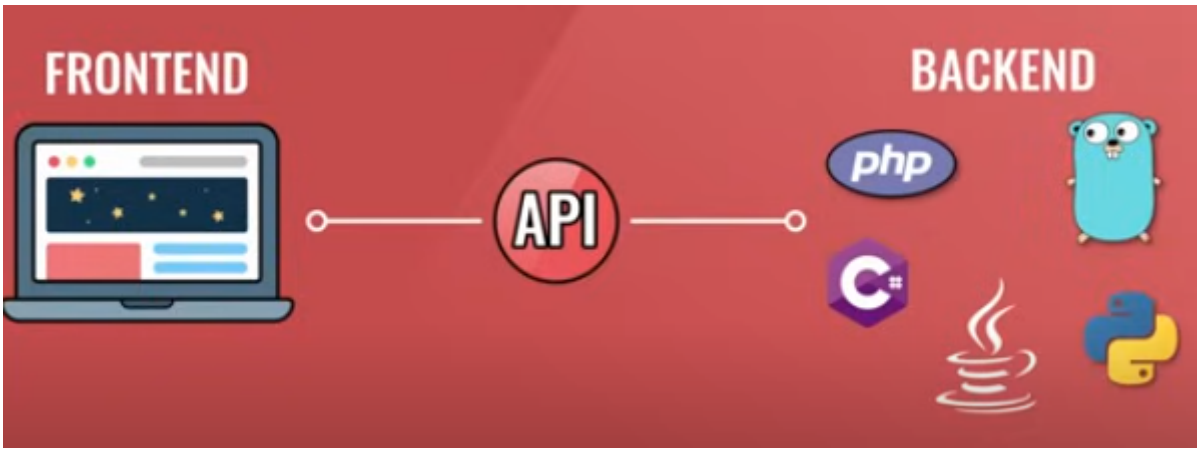
Pero eso cambia alrededor del año 2008-2010 cuando empieza html5 y en ese momento se empieza a hacer la división entre el backend y el frontend. Antes el amo de los navegadores era internet explorer, no respetaba ningún estándar, los programadores teniamos que usar jquery para que las cosas funcionasen, no habia muchas opciones.

Con la llegada de html5, todo cambia, llegan nuevos navegadores como Google Chrome, Firefox y la tenconlogía web crece enormemente.

Html5 no solo es un conjunto de nuevas etiquetas, si no de nuevas APIS que da muchísima potencia al navegador, por lo tanto hay mucha potencia en el cliente y en el servidor.

Antes el cliente era prácticamente un programa para ver, sólo ver y ya un visualizador, no había esa potencia para interactuar y hacer una lógica compleja, a partir de html5 si, por lo tanto se separa el frontend y el backend.

Para conectar el servidor (backend) con el cliente (frontend) se utilizan las APIS, de esa manera un frontend puede funcionar con cualquier backend (da igual que sea php, python, Java, Go, nodejs...)



Lo que hace el frontend es conectarse a los datos a traves de esa API.

Etapas del frontend:

1. Alrededor de 2008-2010 con la llegada de html, css3 y angular.js que es el primer framework imporante declarativo (importa las declaraciones pero no el como se hace). Es fácil de entender las directivas de angular, con ellas le decias si va a ser un ciclo, una validación, imprimir cosas,... Por eso gusto mucho.

```
<button ng-click="count = count + 1" ng-init="count=0">
  Increment
</button>
<span>
  count: {{count}}
</span>

<label>Click me:
<input type="checkbox" ng-model="checked" ng-init="checked=true" />
</label>
<br/>
Show when checked:
<span ng-if="checked" class="animate-if">
  This is removed when the checkbox is unchecked.
```

   Por estos tiempos tambien llega nodejs, mongo, entonces se empieza a hablar de javascript para todo, con nodjs haciamos el backend, con mongo la base de datos, con angular el frontend, entonces se empieza a hablar del **stack mind**, es decir, mongo, express, angularjs, nodejs un stand de tecnologías que se usa para desarrollar.

2. Comienza en 2015 con la llegada de emmascript6 (la sexta versión de javascript), que trae muchisimas funciones nuevas que los programadores estaban esperando, lo que hace un lenguaje mucho más poderoso. da soluciones que antes se hacían con librerías ahora se podían hace con javascript nativo, con esto se deja de utilizar tanto jquery. A partir de la versión 6, empieza a actualizarse cada año a nuevas versiones.

   En 2013 llega react.js que es el segundo gran momento después de angular y poco despues vue.js ya que angular se dividió y se quedo un poco estancado.

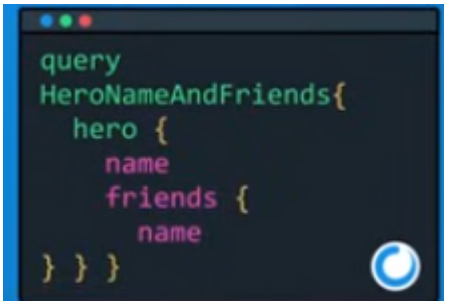
Angular	**Vue.js**	**React.js**
de Google	de la comunidad	de Facebook
Framework: implementa todo el modelo MVC	Framework progresivo: Agrega componentes a medida que tu app necesite.	Libreria: Es solo la vista (V) del modelo MVC.
TypeScript: Usa TypeScript para escribir JavaScript más poderoso	usa **JavaScript** y tiene una curva de aprendizaje muy rápida.	Solo usa **JavaScript** y aprovecha sus características.
Templates. Usa HTML y directivas para insertar la lógica.	Templates y JSX: soporta lo mejor de los 2 mundos: Templates en HTML o JSX.	JSX: Junta la lógica y el mercado en el mismo componente usando JSX.
Two way data binding: Si cambian los datos en la interfaz cambian el modelo (estado).	Two way data binding: La directiva v-model actualiza el modelo si la vista cambia y viceversa.	One way data binding: Si cambian los datos en el estado cambian en la interfaz pero no al revés.

   Aparece también **Graphql**:

   Es un lenguaje de consultas creado por facebook como opción a las API Rest.

   Conceptos bássicos:

   - Query: consulta información al servidor con solo los campos que se necesita (equivalente al método GET de Rest).



   - Mutation: modifican información en el servidor (equivalente a los métodos (POST/PUT/DELETE) de Rest).

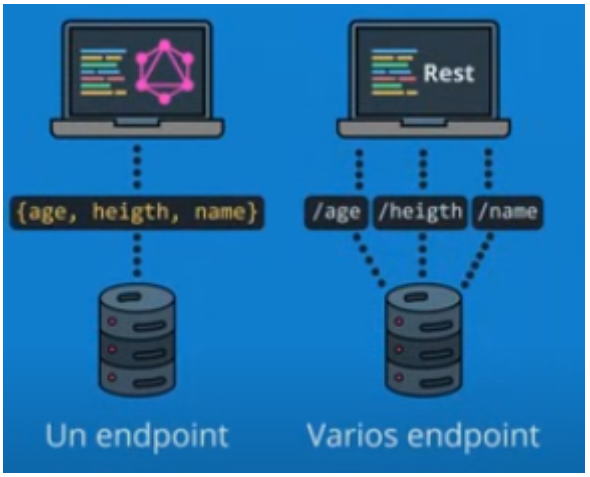
```
mutation CreateReviewForEpisode
  ($ep: Episode!, $rev: ReviewInput!){
    createReview
      (episode: $ep, review: $rev) {
        stars
        commentary
} }
```

Características:

- El frontend puede solicitar datos al servidor sin necesidad de modificar el backend.
- Es una alternativa a las APIs Rest.

En las APIS Rest hay que solicitar a traves de un endpoint que es una url los datos, entonces si necesitas datos de los usuarios, necesitas un endpoint, datos de los cursos, necesitas otro endpoint, si necitas datos de los blogs, otro endpoint y así son muchos, entonces hay que estar solicitándole al backend un point para cada dato que el frontend necesite.

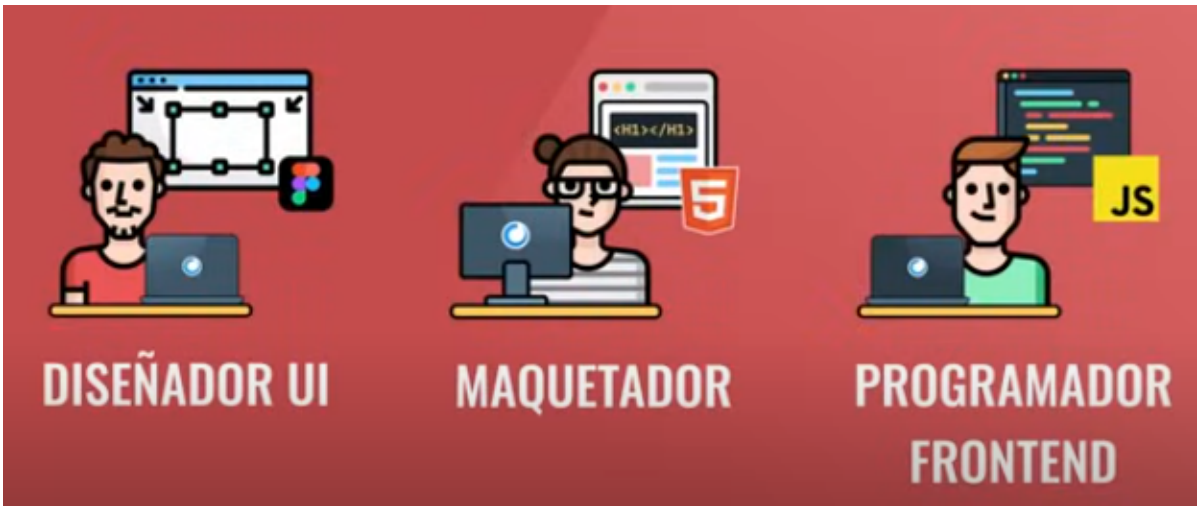
Graphql llega con la idea de que hay un solo end pont, uno solo y a través del él se puedan hacer todas las solicitudes que se necesiten.



3. En el 2018-2019 empezó a hablarse de esto y poco a poco va a tener mucha más fuerza, que está marcada por dos tecnologías:
    1. web assembly que es tener código nativo funcionando dentro del navegador, es decir ya no interpretado (recuerda que hay lenguajes interpretados y lenguajes compilados). Lenguajes como C, C++, ras, .. compilarlos y ese compilado meterlo como una libreria al navegador para que el navegador la ejecute, no la interprete como hace con javascript, con lo cual web assemby permite cosas imposibles de hacer antes con javascript y con mucho mayor rendimiento, pero aun así Javascript sigue siendo el único lenguaje que funciona en el navegador porque, el código compilado lo llevas al navegador.

2. Houdini css que permite meternos en el motor de css que nunca lo podíamos ver, e incluso crear nuestras propias propiedades sin esperar a que lo implementen. Podemos crear nuestros propios estilos y nuestras propias propiedades css metiendonos en el motos a traves de houdini.

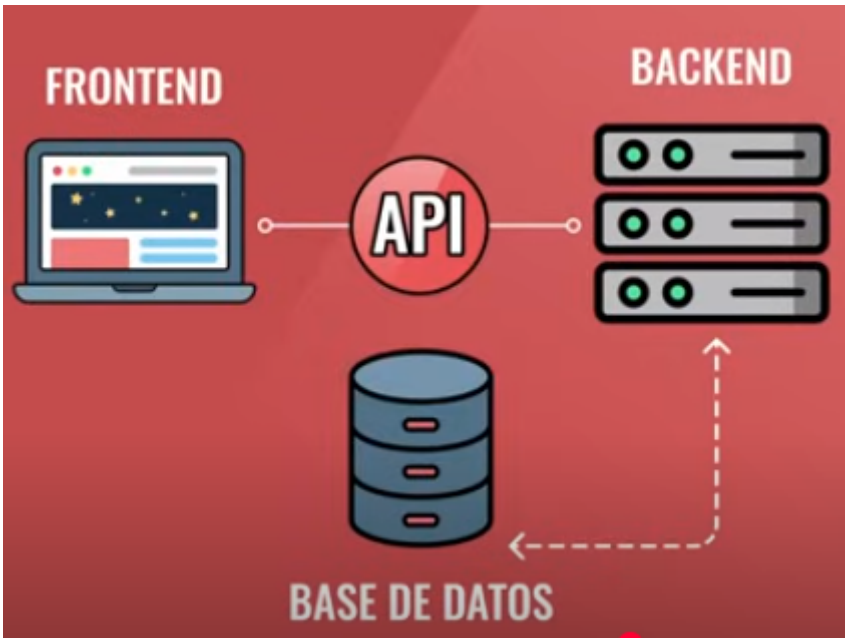
Una confusión en el frontend es que como el frontend es lo que se ve en la pantalla del navegador, la gente asume que un programador frontend debe saber diseñar los colorcitos, los botoncitos,.., pues no un programador frontend no debe por que saber diseñar, para eso están los diseñadores, los maquetadores , que son los que hacen maquetación html css que luego los programadores le añaden la lógica, entonces podemos diferencias tres roles en el frontend:



- Diseñador UI: se encarga de diseñar la interfaz en un programa de diseño como sketch, filmo, adobe xxi
- Maquetador: se encarga de llevar este diseño a código html y css.
- Programador front-end: el que agrega toda la lógica, consume las apis, hace las validaciones, la lógica del lado del frontend.

En el backend, lo que se va a hacer es programar que todo el código que se escriba se va a ejecutar en un servidor, el encargado de la lógica de negocio (todas las funciones que necesita tu cliente, soluciones que se desarrollan para el cliente), por ejemplo si estas haciendo un software de contabilidad, se necesita que se muestren los reportes por mes, o que se organicen por cantidades.

Lo fundamental de un programador backend, es que sepa crear una api, el frontend consume la api, pero el backend es quien debe crearla, en el lenguaje que se quiera. En el desarrollo de la API se va a necesitar conectarse a una base de datos y decidir que datos exponer para que pueda consumir el frontend, es decir un backend tambien tiene que conectarse a una base de datos, el frontend, no se conecta a la base de datos.



Si yo soy backend, necesito saber de base de datos?, técnicamente no, pero es recomendable. Hay soluciones llamadas ORM que nos permiten conectarnos a una base de datos, sin necesidad de saber sql, que es el lenguaje de consultas, pero independientemente de como se haga, la api debe exponer la data para que el frontend la pueda consumir.

El desarrollo backend ha sido tradicionalmente monolítico, es decir un solo backend gigantesco que va creciendo, pero ahora se empieza a utilizar los microservicios, es decir dividir las soluciones en diferentes paquetes e incluso en diferentes servidores, eso tiene muchas ventajas, ya que cada paquete esta aislado del resto, si tienes que corregir algun error, sólamente se corrige ahi sin afectar al resto del backend.

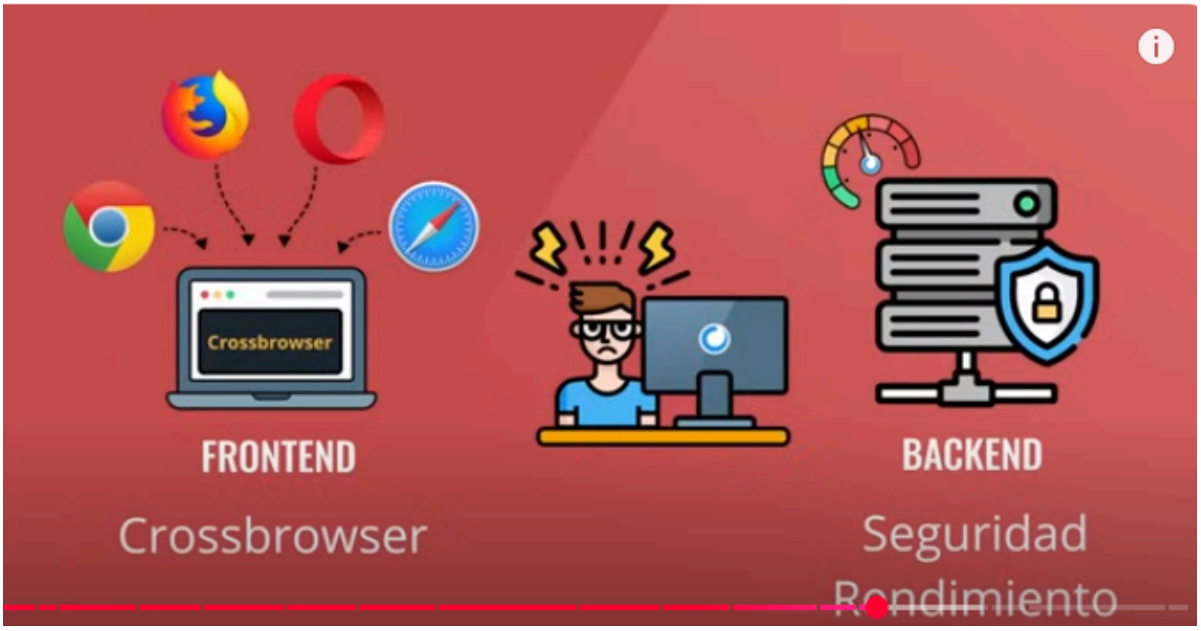
¿Que es la arquitectura de microservicios?

Divide un software en varias aplicaciones pequeñas (microservicioss) que se comunican entre si.

- Podemos usar distintos lenguajes de programación en el mismo proyecto.
- Si un servicio deja de funcionar, no se detiene toda la app, solo el microservicio que falla.
- Podemos actualizar un servicio sin volver a implementar toda la app.
- Los microservicios pueden escalarse de forma indeoendiente.

Los retos de los desarrolladores frontend es que todo su código funciones en todos los navegadores, por que a veces funciona en Chrome pero no funciona en firefox.

Los retos de los desarrolladores backend son principalmente seguridad y rendimiento, rendimiento por el tema de concurrencias, mientras más usuarios hayan más lento se pueden poner los servicios, entonces hay que estar viendo siempre de que rindad bien el sistema y la seguridad, por que, la seguridad del frontend es mas fácil de modificar, asi que es mejor hacerlo en el backend para evitar problemas.



Tener backend de servicio significa tener backend disponible sin necesidad de programadores backend, es decir usar alguna plataforma que nos de soluciones de backend listas, solamente consumas la API, la más conocida hasta el momento es Firebase, por ejemplo para registros de usuario, da base de datos de almacenamiento, entonces simplemente hay que entrar al panel de firebase y configuras lo que se necesite y desde frontend consumes la API de Firebase y ya esta, se puede hacer una aplicación completa sin tener backend en el servidor, ya que está en Firebase.

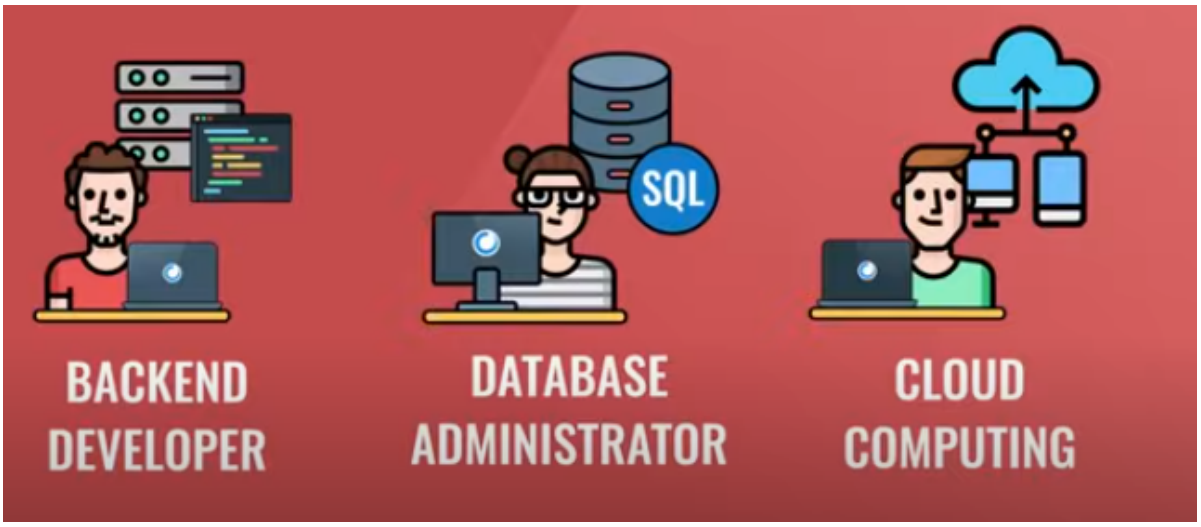
También disponemos de serverlos que son servicios que se consumen sólo cuando sean necesarios, es decir los servidores, por decirlo asi, se encienden cada vez que se necesita el servicio y cuando ya no se necesita se apagan y cuando se necesitan mas de esos servicios de los servidores crecen a demanda, entonces no se necesita un servidor encendido todo el tiempo para un consumo pequeño

También disponemos de servicios en la nube: así como Firebase los proveedores de nube nos dan servicios de backend listos para consumir, por ejemplo un buscador, si necesitas usar la tecnología de Google puedes usar la tecnología de microsoft de bing y crear un buscador en tu sitio web, ya microsoft o Google se encarga de hacer toda la indexación de tu sitio web y te entregan un endpoint que se puede consumir desde el frontend e imprimir un buscador en tu sitio web sin haber echo nada de backend, obviamente luego hay que pagar el recibo por lo que se haya consumido, pero es una tendencia que cada vez toma más fuerza, backend como servicio, y no tener que desarrollarlo dento de tu equipo.

Un desarrollador backend tampoco debe porque saber administrar servidores, ni base de datos, en equipos pequeños no nos queda otra que el backend hago todo eso pero lo ideal es que haya tres roles:

- Backend developer:

- Database Administrator: administrador de base de datos, que es quien diseña y administra la base de datos
- Cloud Computing: administrador de los servidores y del cloud computing, si eres programador backend, no deberías estar instalando certificados ssl.



STACKS:

Es una pila de tecnologías, un conjunto de tecnologías que se utilizan para desarrollar una aplicación, una solución, un software.

Se suele llamar programador fullstack que a la vez hace de backend y de frontend.

El stack mas usado en el desarrollo web tradicional fuel LAMP (Linux, Apache, MySQL y PHP)

Linux: como sistema operativo en el servidor

Apache: como servidor web

MySQL: como base de datos

PHP: como lenguaje de programación

El stack mean es :

Mongo: para la base de datos.

Express: como framework de nodejs

Angularjs: como framework para la parte del frontend

Nodejs: para el backend

Stack mern: que simplemente reemplaza a angular por react.js

Stack de EDteam:

Linux: sistema operativo del servidor

Postles: como base de datos

GO: lenguaje de backend

ego: como framework de go

Nodejs: para el server de rendering

Next: para el server de rendering.

React: para el frontend

Edgris: para los estilos.

Es muy dificil ser programador fullstack, por que es prácticamente imposible saber de todo, ya que en cada empresa por ejemplo pueden utilizar unas u otras tecnologías.

En el frontend: programación en el cliente, hay un solo lenguaje de programación JavaScript, pero varias tecnologías (angular, vue, react).

En el backend: hay muchos lenguajes de programación que se pueden usar: Java, Go, Python, PHP, ..

https://www.youtube.com/@EDteam

Puedes encontrar más información en los siguientes recursos:

- Chrome for Developers. Recuperado el 30 de abril de 2024 de https://developer.chrome.com/?hl=es-419.
- Microsoft Edge Developer. Recuperado el 30 de abril de 2024 de https://developer.microsoft.com/es-es/microsoft-edge/?form=MA13LH.
- Fazt. Modelo Cliente Servidor, Explicación Simple. Recuperado el 21 de febrero de 2024 de https://www.youtube.com/watch?v=49zdlyLSwhQ&t=1s
- John Bonnemaison. Arquitectura Cliente Servidor. Recuperado el 21 de febrero de 2024 de https://www.youtube.com/watch?v=5-l7p7apV3I
- EDteam. ¿Qué es BACKEND y FRONTEND? Recuperado el 21 de febrero de 2024 de https://www.youtube.com/watch?v=50RbVujPPGs

RETO:

Inspeccionar y modificar una página web con DevTools.

Tu misión es utilizar las herramientas de desarrollador en el navegador que prefieras. Elige una página web sencilla, como Wikipedia. Abre la herramienta de inspección. Te doy una pista: puedes encontrarla con la tecla F12 o haciendo clic con el botón derecho del ratón y eligiendo “Inspeccionar”.

Una vez has abierto DevTools, tu misión es:

- Seleccionar un elemento en la página, como el título principal.
- Observar en el panel de elementos cómo el HTML se resalta al seleccionar diferentes partes de la página.

- Modificar el texto de un elemento. Por ejemplo, cambiar el título principal de la página por "Mi primera inspección".
- Cambiar el color de fondo o el color del texto de un párrafo utilizando el panel de estilos CSS.

2. HTML-Lenguaje de marcas para la estructura de contenido

2.1 Estructura básica de un documento HTML.

Introduccion

HTML es el lenguaje fundamental que forma la base de todas las páginas web. A través de HTML podemos crear páginas de contenido estructuradas que cualquier navegador podrá interpretar.

HTML (Hypertext Markup Language) o Lenguaje de Marcado de Hipertexto. Utiliza "etiquetas" o "elementos" para marcar diferentes partes del contenido.

Hace un par de décadas algunas personas decían que, si no estabas en la web, no existías. Bueno, imagínate ahora. En dos décadas se pasó de crear páginas web, básicamente estáticas, diseñadas a mano, escribiendo cada línea de código, a la existencia de herramientas web, sistemas de gestión de contenido, redes sociales y muchas cosas más.

Pero la base es la misma: el lenguaje HTML.

EXPLICACIÓN

A través de HTML podemos crear páginas de contenido estructuradas que cualquier navegador podrá interpretar, ¡incluso los de los móviles! Permite incorporar imágenes, enlaces, listas y formularios. Además, HTML ha seguido evolucionando. El desarrollo HTML5 ha supuesto una mejora en la accesibilidad y la experiencia de usuario. Te invito a hacer una comprobación. Abre cualquier red social en el navegador del ordenador y busca en el menú de los tres puntos arriba a la derecha, “Herramientas de desarrollo”. Verás algo como esto:

```
<!DOCTYPE html>
<html id="facebook" class="_9dls_fb-light-mode" lang="es" dir="ltr">
<head></head>
<body class="_6s5d
71pn system-fonts--body segoe">== $0
▶<div id="mount_0_0_xb" style>...</div>
<script type="application/json" data-content-len-"91" data-sjs> </script>
<script
type="application/json" data-content-len-"53" data-sjs>{"require":
[["maybeDisableAnimations",null,null,[]]]}</script>
<script type="application/json" data-content-len-"49" data-sjs>{"require":
[["replaceNativeTimer",null,null,[]]]}</script>
<script type="application/json" data-content-len-"60" data-sjs>{"require":
[["bootstrapWebSession",null,null,[1788545762]]]}</script>
<script type="application/json" data-content-len-"220" data-sjs>
</script>
▶<script type="application/json" data-content-len-"158" data-sjs> </script>
```

Sí, HTML es la base de todo lo que hay en la web. Profundiza en este lenguaje del que todos nos beneficiamos con mucha frecuencia.

HTML también juega un papel vital en la optimización de motores de búsqueda (SEO), ya que las etiquetas y la estructura semántica del contenido ayudan a los motores de búsqueda a entender y clasificar el contenido de las páginas web.

Historia de HTML

HTML fue creado por Tim Berners-Lee en 1990 mientras trabajaba en el CERN y fue diseñado para **facilitar el intercambio de información científica y académica entre investigadores.** Desde su creación, HTML ha pasado por varias revisiones y actualizaciones, hasta llegar a HTML5, que es la versión actual y la más avanzada, incorporando soporte para multimedia, gráficos, efectos interactivos y compatibilidad con dispositivos móviles.

HTML es fundamental para el desarrollo web porque proporciona la **estructura básica de todas las páginas web**. Sin HTML, no podríamos organizar el texto, crear enlaces, insertar imágenes o videos, ni realizar muchas otras funciones que damos por sentadas en las páginas web modernas.

Estructura básica de una página web

Cada documento HTML sigue una estructura estándar, que asegura que los navegadores puedan interpretar y mostrar el contenido de manera correcta a los usuarios.

Los **componentes básicos** que lo constituyen: los **elementos** y las **etiquetas HTML**. Estos son los bloques de construcción fundamentales de cualquier página web, permitiendo a los desarrolladores definir la estructura, organizar el contenido y especificar la presentación de la información en el navegador.

Esta estructura básica incluye varios elementos esenciales que trabajan juntos para definir el encabezado (head) y el cuerpo (body) de la página, entre otros componentes. Aprender a estructurar correctamente un documento HTML es el primer paso crítico para cualquier desarrollador web, ya que establece la base sobre la que se construirán estilos con CSS y se añadirán funcionalidades con JavaScript.

Definición y sintaxis

- **Definición**: En HTML, un elemento consiste en una etiqueta de apertura, que contiene el contenido, seguida de una etiqueta de cierre. Las etiquetas son indicadores que le dicen al navegador cómo debe procesar y presentar el contenido entre ellas. Por ejemplo, un elemento de párrafo se representa mediante las etiquetas

  (apertura) y

  (cierre), y el texto que se coloca entre estas etiquetas se muestra como un párrafo en la página web.
- **Sintaxis**: La sintaxis de una etiqueta HTML típica incluye el nombre de la etiqueta encerrado en corchetes angulares (< >).
  - Cuando una etiqueta requiere un cierre, se utiliza una barra inclinada (/) antes del nombre de la etiqueta en la etiqueta de cierre.

- Algunos elementos, conocidos como elementos vacíos (por ejemplo, la etiqueta de imagen y la etiqueta de salto de línea < br>, no contienen contenido y por lo tanto no requieren una etiqueta de cierre.

Elementos de apertura y cierre

La mayoría de los elementos HTML se componen de una etiqueta de apertura y una etiqueta de cierre, con el contenido relevante ubicado entre ellas. La etiqueta de apertura marca el inicio del elemento, mientras que la etiqueta de cierre señala el final. Este par de etiquetas define el alcance del elemento en el documento. Por ejemplo, en < h1> encabezado principal ,

es la etiqueta de apertura y

es la etiqueta de cierre, y juntas envuelven el texto del encabezado principal.

En contraste, los elementos vacíos o auto-cerrables no encierran contenido y por lo tanto no requieren una etiqueta de cierre. Estos elementos a menudo incluyen atributos para especificar su comportamiento o presentación. Un ejemplo clásico es la etiqueta de imagen), que utiliza atributos como src (para especificar la URL de la imagen) y alt (para proporcionar texto alternativo en caso de que la imagen no pueda mostrarse).

< !DOCTYPE html>

Define el tipo de documento y la versión de HTML. Ayuda al navegador a entender qué versión de HTML está siendo utilizada para interpretar correctamente el contenido de la página. En HTML5, la última versión de HTML, esta declaración es simplificada a:

Elemento con los atributos lang

Después, pondrías el elemento , actúa como el contenedor raiz de todo el contenido de la página.

El atributo lang dentro del elemento es importante para definir el idioma principal del documento, lo que ayuda a los motores de búsqueda y las tecnologías de asistencia a entender y procesar el texto adecuadamente. Por ejemplo, indica que el idioma principal del documento es el inglés

Secciones y

Dentro del elemento , la estructura se divide en dos secciones principales: y .

- Sección : Esta sección contiene metadatos sobre el documento, que no son visibles directamente en la ventana del navegador pero son esenciales para la definición de la página y la comunicación con los navegadores y motores de búsqueda. Elementos comunes dentro del incluyen